

(Programmers will often use the word *subclass* as a verb, as in “Let’s subclass `classA` to build `classB`.”) Here’s an example with real class names:

```
class Widget():
    # Widget's methods

class WidgetWithFrills(Widget):
    # WidgetWithFrills's methods
```

The `Widget` class will provide general functionality. The `WidgetWithFrills` class will include everything from the `Widget` class and define any additional methods and instance variables it wants with more specific capabilities.

Employee and Manager Example

I’ll start with an extremely simple example to make the key concepts clear, then move on to some more practical examples.

Base Class: Employee

[Listing 10-1](#) defines a base class called `Employee`.

File:

EmployeeManagerInheritance/EmployeeManagerInheritance.py

```
# Employee Manager inheritance
#
# Define the Employee class, which we will use as a base class

class Employee():
    def __init__(self, name, title, ratePerHour=None):
        self.name = name
        self.title = title
        if ratePerHour is not None:
            ratePerHour = float(ratePerHour)
        self.ratePerHour = ratePerHour

    def getName(self):
        return self.name
```